NOVA SCHOOL OF SCIENCE & TECHNOLOGY

Internet Applications Design and Implementation

2023/2024

Online Renting Service



Authored by:

57502 – David Delgadinho

60665 – Renato Viola

52949 – Romão Costa

NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

Introduction

In this assignment, we were tasked with developing an application for an apartment management and renting system.

It was composed by two stages: the initial phase was exclusively dedicated to designing and building the API to then be used in the latter phase, which mainly regarded the implementation of *JWT* token-based security and the client-side of the system.

For the purpose of this report, we'll be mostly focusing on the work involved in the frontend of the application.

Our primary objective was to provide a intuitive, interactive and user-friendly interface for different types of users – *Clients*, *Owners* and *Managers* - that is both functional and safe to use. By adopting the *React*/*Redux* framework and *Typescript*, we had a strong toolkit with which to ensure those requirements.

Through this interface, users can engage in various activities such as browsing apartments, make reservations and manage the states of such apartments and reservations, among other functionalities described in the user stories we were given.

Over the course of this report, we will strive to provide a comprehensive overview of the design and implementation decisions taken, as well as noting the challenges encountered throughout the development process.

Design

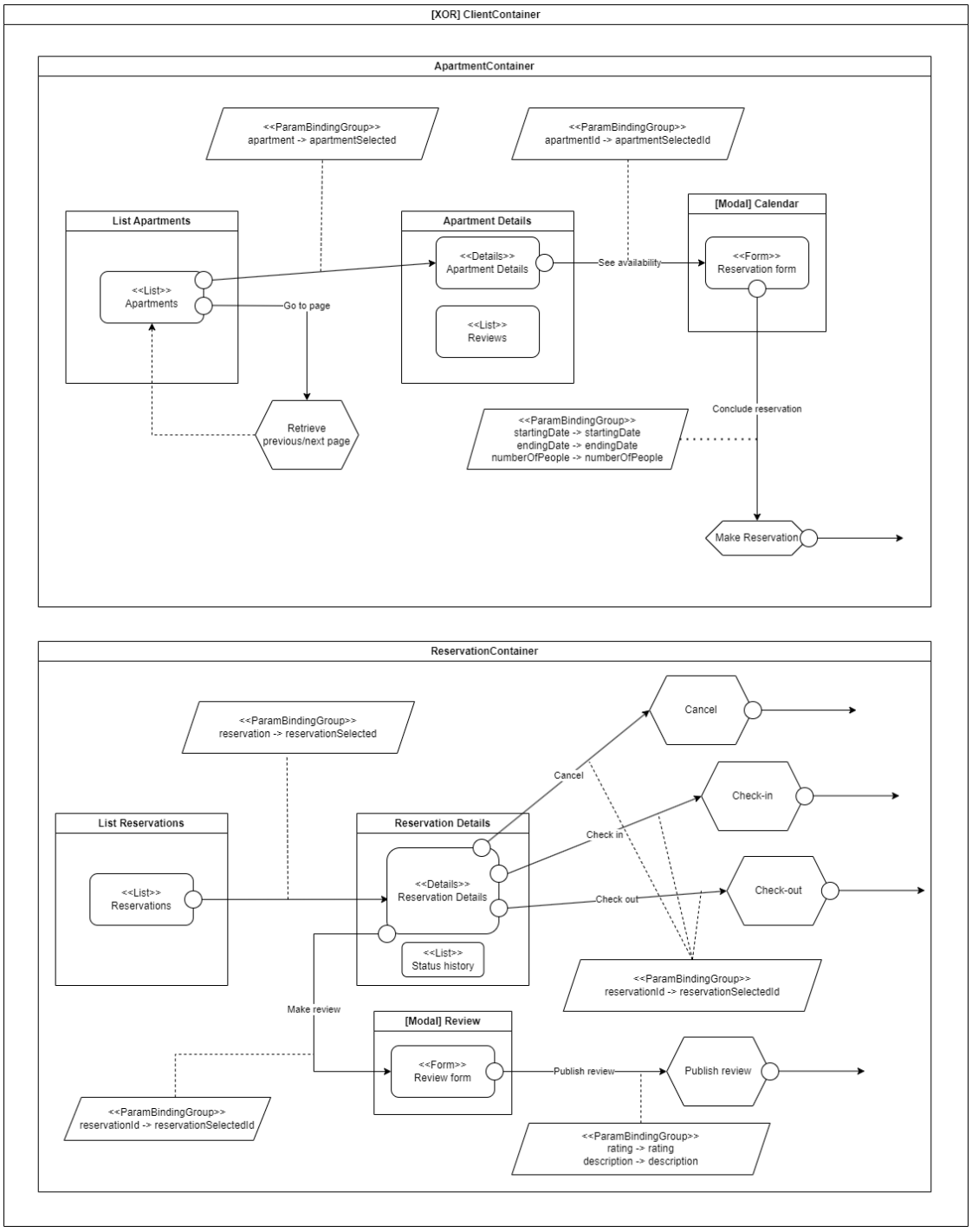
Jumping straight into coding without a clear direction in mind would likely end poorly and far from our goal of creating a fluid and user-friendly environment.

As our aim for this project aligns with the core philosophy of *IFML – user interaction*, we utilized this modeling language to create an intuitive visual model of how users with different permissions interact with the various components of our application, in accordance with the established user stories. This process allowed us to craft interfaces and interactions tailored to the needs and expectations of users, whilst defining the data flow taking place among components.

There are three types of users in our system: *Client*, *Owner* and *Manager*. Due to time constraints and not being mentioned in the user stories, a Manager does not have its own view of this application. However, the users of the remaining types do have their own view of the interface, restricting their access to the operations and resources, depending on the permissions their type of account imply.

In light of not only the aforementioned time constraints but also the small scale of the user stories and the similarities between them, we chose not to model these individually and instead modelled the general panorama of the application, with a part designated specifically for *Clients* and another exclusively for *Owners*.

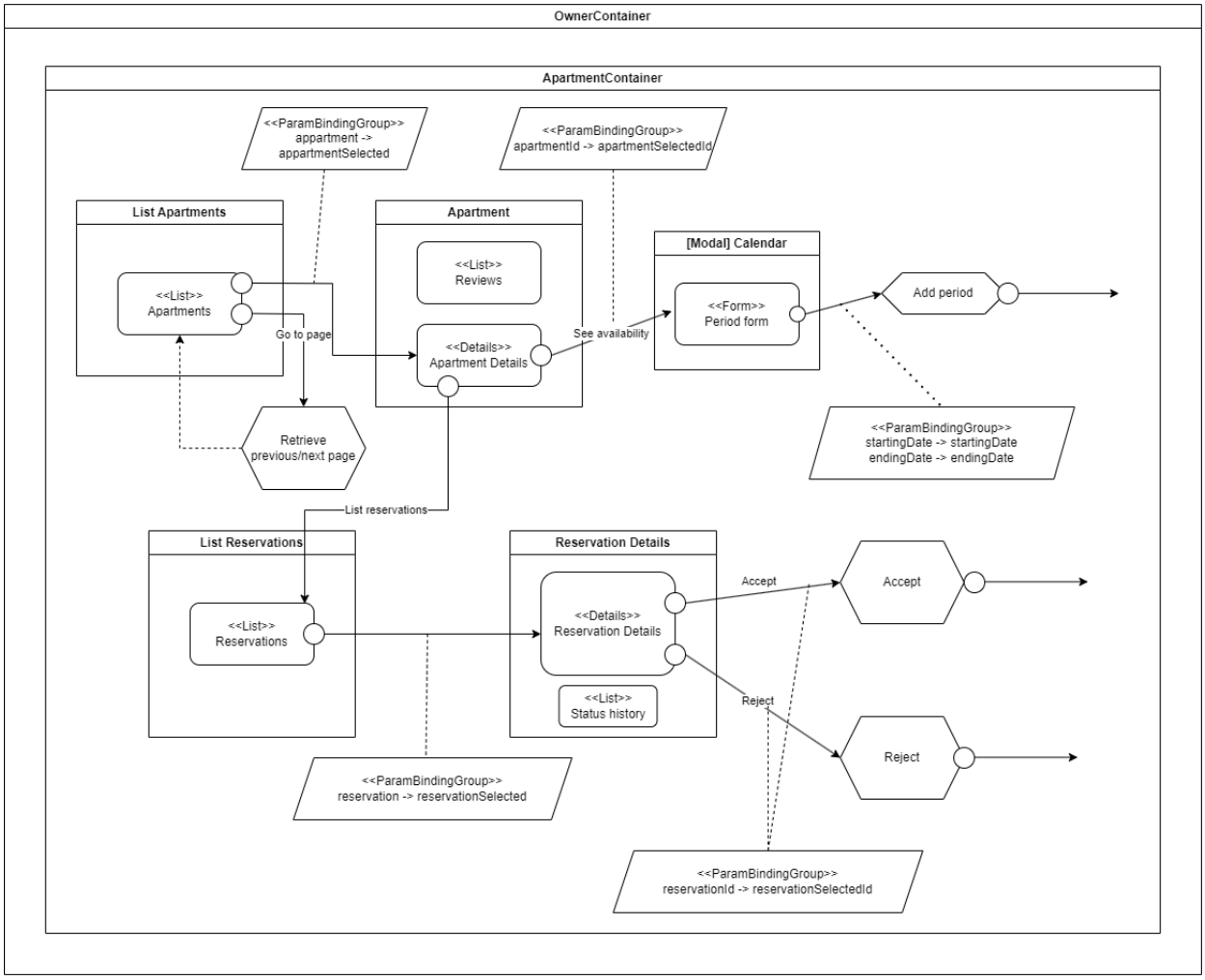
Client View



As displayed in the diagram above, a user of type ***Client*** cannot simultaneously interact with apartments and reservations. Instead, let us address what a ***Client*** can do:

- *Browse the list of all apartments in the system up for rental, page by page. Selecting one will redirect the user to a page displaying further details about it, as well as its reviews. At this point, the user may check the availability of such apartment and make a reservation by filling out a form.*
- *Browse the list of its own reservations. Selecting one redirects the user to a page displaying further information about it, as well as its history of statuses. From here, depending on the current status of the reservation, the user may perform actions such as: cancelling it; checking in; checking out; leaving a review (by filling out a form).*

Owner View



Unlike a *Client*, a user of type *Owner* will not have access to all the apartments in the system, only the ones it owns.

It also won't have its own reservations, so to speak. Instead, it has access to the reservations made to its apartments.

An *Owner* may browse the list of all the apartments it owns, page by page. Selecting one will redirect the user to a page displaying further details about it, as well as its reviews.

At that point the user may choose to:

- *Add a period of availability to the selected apartment by filling out a form.*
- *Be redirected to the list of reservations made to the selected apartment.*

If the user chose to access the reservations of an apartment, then:

- *It may browse them at will.*
- *Selecting a reservation redirects the user to a page displaying further information about it, as well as its history of statuses.*
- *From this point, depending on the current status of the reservation, the user may perform actions such as: rejecting it; accepting it.*

Implementation

Nearly all of the system's logic is handled by the *REST* API developed in the first phase of the assignment using the *Spring* framework and *Kotlin*. Having documented it with *OpenAPI 3.0*, it is here that we expose all the endpoints allowing the manipulation of the data referent to the different entities of the system.

In order to meet the requirements of the user stories disclosed in the second phase, some minor adjustments to the backend were in order.

A notable implementation change was the introduction of pagination, where we adapted the endpoints previously returning a list of apartments to now return a page of apartments with a fixed size. With this solution, only the apartments of the current page in the client-side need to be loaded from the server, making for a more scalable and efficient system.

That aside, the most crucial aspect of the changes we made to our server-side was the transition from basic authentication to a *JWT* authentication mechanism. In the frontend, after successfully logging in, the user is redirected to a specific area where only operations permitted to its role are accessible.

These authorization checks, combined with the token-based authentication mechanism implemented in the backend further reinforce the security of our system, ensuring that users do not overstep their permissions.

The conception of the application in the second phase was driven by user stories, ensuring that features were developed in response to real user needs and scenarios. This user-centric approach facilitated the creation of the client-side application to complement the server-side logic.

For the client-side of our application we used *React* with *Typescript* alongside *Redux*. To the best of our abilities, we tried to follow *React's* guidelines, by splitting the contents of pages into re-usable components where we could, carefully using the different hooks, props and event handlers. *Typescript's* strict typing helped us make our code more reliable and maintainable.

While the UI and component layout was handled with *React*, the application's state was managed with *Redux*, as we had slices for apartments, reservations and even the user itself, where we defined the actions for fetching and updating this data, immediately reflecting in the UI.

To speed up and facilitate developing the user interface, while holstering a modern and sleek design, we made use of the framework *Tailwind CSS* and external libraries such as *Material UI*, *Daisy* and *react-modern-calendar-datepicker* for the implementation of the calendar needed for the app itself.

In this phase, our priority was the correct implementation of the given user stories over the application's aesthetics. Upon asserting the functionality of our solution through some testing, only then we turned our attention to styling the components more attentively.

It was in the final stages of development that our focus shifted over to the visual presentation of the frontend.

Challenges during development

Communication restrictions due to CORS policy

Initially, performing requests from our client-side application to the API wasn't as straightforward as we had expected. Since our *React* app and Spring backend are served in different domains, our requests were considered cross-origin requests, thus getting blocked by the *CORS* policy. To solve this issue, we had to enable *CORS* in the server-side by adding a configuration to allow requests from the client-side's origin. Through some time searching and *GPT-4's* magic, the problem was handled.

Error when unmounting Calendar component

One critical issue that we were stuck with for several days was related to the *Calendar* component from the *react-modern-calendar-datepicker* library. Whenever this component was removed from the DOM, it would cause "*TypeError: Cannot read properties of null (reading 'removeEventListener')",* crashing the page and forcing us to reload it. Many hours were wasted thinking it was something in our code causing it, but after thorough search, it turned out that many other people using this component had the exact same problem. The problem was in the library itself and fortunately a Github user by the name of *mojabyte* published his fix, so the package we are actually using is his version of *react-modern-calendar-datepicker*, not the original.

Redux wasn't easy to grasp initially

The incorporation of *Redux* in our *React* application proved to be quite challenging as understanding not only the concepts of actions, reducers and slices, but also the overall Redux flow was time-consuming and required significant effort, which made its implementation more difficult.

Inexperience in styling

Due to some inexperience in the area, styling and enhancing the user interface's visual was very demanding. At first, we tried to balance the aesthetics of the application with functionality, but quickly concluded that this approach wouldn't go far, which resulted in, as we said earlier, the team initially focusing on mostly logic and functionality, leaving the visual elements of the application to the last stages of development.

Struggling for time

The different schedules of the team's members and imposed time constraints made it difficult to create individual *IFML* diagrams for each user story. In the beginning of the second phase we developed a prototype for the overall *IFML* diagram, one which we continuously kept building upon until reaching the state presented in the *Design* section, but skipped over the individual diagrams in order to start developing the client-side application as quickly as possible. In the end, we couldn't arrange the time to develop a diagram for each user story.

Conclusion

Over the course of both phases of the assignment, our team developed a fully working and competent web application with a React/Redux frontend and a Spring backend. Users may enjoy the intuitive and easy-to-use interface we've designed, while performing the typical operations expected from a service of this kind.

We do believe our solution would benefit from some further improvements, namely in its visual presentation and mostly code organization, as we each had varying skills and our own approach to programming, we could not find the time to conciliate on a coherent way to implement the various components.

While acknowledging the room for improvement in our implementation, we are very pleased with the results of our work. Our application delivers the interactive and safe user experience we set out to achieve, fulfilling all the required user stories.

We found this assignment very interesting albeit challenging. The modern technologies we got to work with enabled us to come out of our comfort zone and demanded much effort.

Relevant notes

Use of GPT-4

Throughout the assignment, *ChatGPT* was an essential resource to us. In an environment where time was of the essence, its assistance significantly sped up our development process.

It was particularly useful in correcting minor styling mistakes and handling tedious tasks such as producing the seed data for our server, but what we used it for the most was explaining and giving further context to the errors we'd get. One specific case where *GPT's* assistance was invaluable was in better understanding and configuring the *CORS* policy in the server-side.

Though *GPT* does make mistakes quite often, the way it exposes and contextualizes what is requested would many times improve our awareness of what we should be searching for, even if the response was incorrect. We found it to be an amazing and very practical learning tool, despite it being a good idea to fact check nearly all the information it provides.

Project Repository

As soon as we had the initial setup for what our client application would eventually be, we tried uploading it to the project's repository, placing it in a folder called *Client* and the repository's old contents in a folder named *Server*. This process ended up interfering with *Intellij's* configurations for the backend, and we couldn't even run it anymore.

Not wanting to waste anymore time, we rolled back those changes and postponed merging the phases. As a temporary measure, the development of the frontend took place in the private repository of one of the team's members, until recently merging it with the main repository. We also had to give up on the idea of having the backend in its own folder.

By now, the professor has been invited to the mentioned repository, which can be found here.